



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

Materia: Desarrollo de Juegos Interactivos

Elemento de Evaluación: Proyecto Bimestral

Integrantes:

Josue Peralta

Ariel Sanchez

Nombre del Proyecto:

Diseño de Videojuegos, Arquitectura y Análisis

TERCER ENTREGABLE

ARQUITECTURA TÉCNICA (DISEÑO SIN IMPLEMENTACIÓN)

1. Objetivo

Simular la fase profesional de pre-producción y diseño técnico de un videojuego, definiendo una arquitectura mantenible y escalable sin implementación. Se especifican decisiones tecnológicas, patrones de diseño, estructura de módulos, flujos de eventos y diagramas UML que permiten iniciar una fase de producción con mínima ambigüedad.

2. Stack Tecnológico Propuesto

Para un proyecto académico y de prototipado con enfoque en ingeniería, se plantea un stack estándar de motores modernos. La selección final depende del alcance (2D/3D) y experiencia del equipo.

Categoría	Propuesta	Justificación
Motor	Unity (recomendado) / Godot	Ecosistema amplio, soporte a patrones, tooling y exportación multiplataforma.
Lenguaje	C# (Unity) / GDScript o C# (Godot)	Tipado/estructura para diseño orientado a componentes y sistemas.
Datos	JSON (configuración data-driven)	Balanceo y contenido editable sin recompilar; trazabilidad en Git.



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

Guardado	SaveGame.json (progreso y configuración)	Persistencia simple, portable y auditable.
Control de versiones	Git + GitHub	Historial de cambios y evidencia de trabajo; colaboración.

3. Filosofía de Arquitectura

Echoes of Control prioriza composición sobre herencia y separación de responsabilidades. El diseño se organiza en sistemas (lógica global), componentes (capas reusables en entidades), y managers (orquestración). La comunicación entre sistemas se realiza mediante eventos (Event Bus) para minimizar acoplamiento, permitiendo que UI/Audio reaccionen al gameplay sin depender de clases específicas.

4. Estructura de Módulos y Carpetas (Propuesta)

La estructura del proyecto define convenciones claras para evitar deuda técnica y mantener localización de responsabilidades. Se propone una organización por capas (core, systems, components, ui, data), aplicable a Unity/Godot.

/src

/core

GameManager

SceneLoader

InputManager

EventBus

/systems

ThreatSystem

ResourceSystem

EventSystem

ProgressionSystem

SaveService

/components



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

HealthComponent
StabilityComponent
TimerComponent
/ui
UIManager
HUDWidgets
/audio
AudioManager
MusicLayerController
/data
DataRepository
configs/
 subsystems.json
 events.json
 difficulty.json
/utils
Constants
MathHelpers

5. Patrones de Diseño Seleccionados y Justificación

Los patrones se seleccionan para responder a problemas típicos de videojuegos: control de flujo, desacoplamiento y creación de objetos data-driven.

Patrón	Dónde se aplica	Justificación
State Pattern	GameManager (MainMenu/Playing/Paused/Results) y SystemStateMachine (Stable/Unstable/Critical)	Evita condicionales extensos y hace explícitas las transiciones; facilita tuning y pruebas.
Observer / Event Bus	ThreatSystem, EventSystem publican eventos; UIManager/AudioManager se suscriben	Desacopla sistemas de UI/Audio; reduce



**ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN**

		dependencias y mejora mantenibilidad.
Factory	ActionFactory / EventFactory (instanciar acciones/eventos desde JSON)	Centraliza creación; permite contenido data-driven; minimiza duplicación.
Component Pattern	Entidades (Jugador/Sistema) con componentes (Stability, Timers, Health)	Reutilización sin explosión de herencia; permite variantes por configuración.
Repository	DataRepository para cargar configs JSON y exponerlas tipadas	Aísla acceso a datos; facilita pruebas y cambio de fuente de datos.

6. Diseño Data-Driven (JSON)

La configuración de subsistemas, eventos y dificultad se define de forma data-driven. Esto permite ajustar balance y comportamiento sin tocar lógica. Los JSON se versionan y se documentan.

6.1 Ejemplo: subsystems.json

```
[  
  { "id": "core", "name": "Sistema Central", "stabilityMax": 100, "regenPerMin": 1.0 },  
  { "id": "grid", "name": "Red de Energía", "stabilityMax": 80, "regenPerMin": 0.5 },  
  { "id": "comm", "name": "Comunicaciones", "stabilityMax": 70, "regenPerMin": 0.3 }  
]
```

6.2 Ejemplo: events.json

```
[  
  { "id": "overload", "name": "Sobrecarga", "severity": 2, "targets": ["grid"], "timeToResolveSec": 30,  
    "stabilityDelta": -10 },  
]
```



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

```
{ "id": "signal_loss", "name": "Pérdida de señal", "severity": 1, "targets": ["comm"],  
"timeToResolveSec": 20, "stabilityDelta": -6 }  
]
```

7. UML – Diagramas Requeridos

Los siguientes diagramas UML representan la arquitectura sin implementación. Se incluyen (1) diagrama de clases, (2) casos de uso, (3) máquina de estados del flujo del juego y (4) máquina de estados del sistema global. Las figuras se insertan como imágenes en los espacios reservados.

FIGURA 1

Este diagrama representa la arquitectura lógica del videojuego a nivel de clases y sistemas principales. Se observa una organización modular basada en sistemas independientes (ThreatSystem, ResourceSystem, EventSystem, ProgressionSystem) coordinados por un GameManager central. La comunicación entre módulos se realiza mediante un EventBus, aplicando el patrón Observer para reducir el acoplamiento entre lógica de juego, interfaz y audio. El diseño prioriza la composición sobre la herencia y permite una arquitectura escalable, mantenible y alineada con buenas prácticas de ingeniería de software.

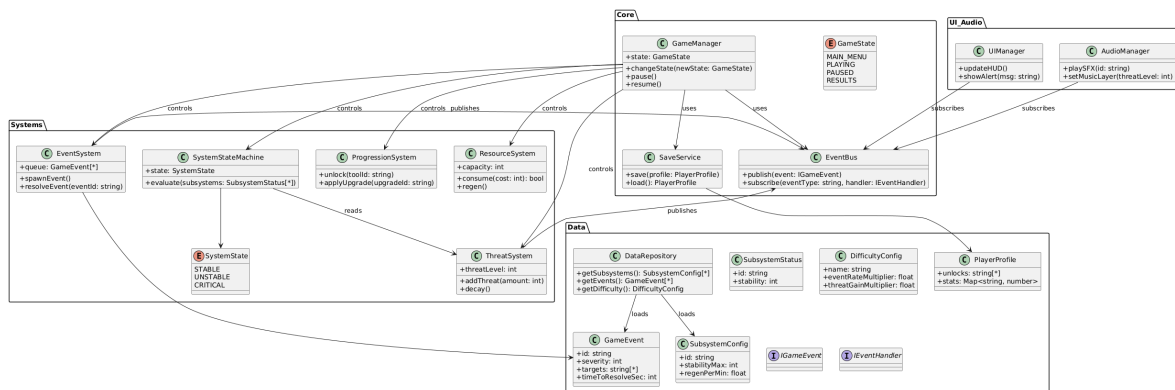


FIGURA 2

El diagrama de casos de uso describe las interacciones principales entre el actor Jugador y el sistema. Se identifican las acciones fundamentales que conforman el gameplay, tales como analizar el estado del sistema, asignar recursos, resolver eventos y gestionar la amenaza. Los casos de uso reflejan un enfoque centrado en la toma de decisiones estratégicas, donde varias acciones incluyen



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA DE SISTEMAS INFORMÁTICOS Y DE COMPUTACIÓN

implícitamente la gestión del nivel de amenaza, reforzando la relación entre gameplay y presión sistémica.

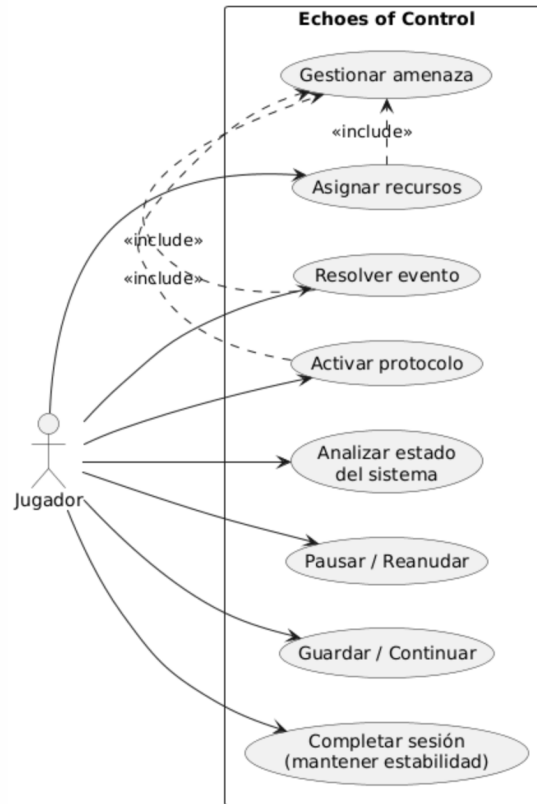
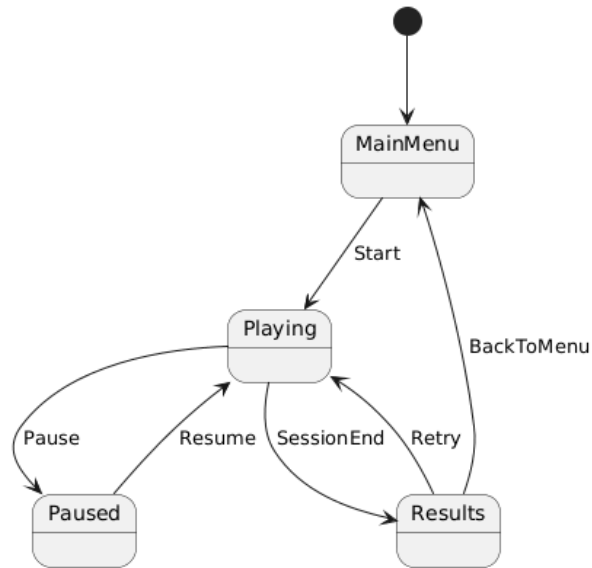


FIGURA 3

Este diagrama muestra los estados principales del videojuego y las transiciones posibles entre ellos. El flujo inicia en el menú principal, desde donde el jugador puede comenzar una sesión de juego. Durante la partida, el sistema permite pausar y reanudar la ejecución, así como finalizar la sesión y acceder a los resultados. Este modelo de estados aplica el patrón State, facilitando el control del flujo del juego y evitando dependencias excesivas entre la lógica de control y las escenas.



FIGURA 4



El diagrama representa la evolución del estado del sistema controlado por el jugador, modelado mediante tres estados principales: Stable, Unstable y Critical. Las transiciones entre estados dependen de variables como el nivel de amenaza y la estabilidad promedio de los subsistemas. Este modelo permite formalizar la dificultad dinámica del juego y refuerza la experiencia de tensión y responsabilidad, ya que las decisiones del jugador influyen directamente en la progresión hacia estados más críticos.

